

From Universal Denoising to Soft Mixture-of-Experts Restoration and Style-Conditioned Face-to-Sketch Synthesis: Design, Optimisation and Deployment of Four Generative Systems

Muhammad Ibrahim Malik (23i-0694)

Department of Computer Science, FAST-NUCES Islamabad, Pakistan

Course instructor: Dr. Akhtar Jamil

Generative AI — Assignment 1

Code: <https://github.com/ibrahimmlk/Generative-AI-Assignment-1>

Demo video: <https://youtu.be/uYxdYkkurZU>

Abstract—We design, train, evaluate and deploy four related generative systems. Tasks 1–3 restore 128×128 Oxford-IIIT Pet images corrupted at run time by salt-and-pepper noise, Gaussian blur or rectangular occlusion: (1) a single universal convolutional denoising autoencoder (DAE) with a genuine latent bottleneck, (2) a hard-routed system in which a CNN corruption classifier selects one of three specialist DAEs, and (3) a differentiable soft mixture-of-experts (MoE) whose gate is initialised from the classifier and trained jointly with the experts under reconstruction, SSIM, cross-entropy and routing-balance losses. Task 4 is a style-conditioned pix2pix GAN that converts FS2K face photographs into sketches in one of three styles using a learned style embedding injected into both generator and discriminator. Every model was tuned with Optuna (TPE sampler, median pruning, routing-collapse pruning for the MoE), tracked with Weights & Biases, exported to ONNX (max. deviation from PyTorch $\leq 2.0e - 06$) and served through a React/Tailwind front end and FastAPI back end started by a single Docker Compose command. On the official test set the MoE reaches 26.46 dB / 0.838 SSIM averaged over all corrupted inputs, compared with 25.99 dB / 0.817 for the universal DAE and 26.12 dB / 0.808 for predicted hard routing.

Index Terms—denoising autoencoder, image restoration, mixture of experts, conditional GAN, pix2pix, Optuna, ONNX

I. Introduction

Real images are degraded in many different ways, and a practical restoration system rarely knows which degradation it faces. Three design philosophies exist: a single universal model that must internalise every degradation, a hard-routed pipeline that first recognises the degradation and then calls a specialist, and a soft mixture that blends specialists with input-dependent weights. We implement all three on the same data, the same corruption definitions and the same evaluation protocol, so their behaviour can be compared directly, and complement them with a paired image-to-image GAN for face-to-sketch synthesis. The contributions of this report are: (i) a reproducible run-time corruption pipeline with deterministic validation/test

manifests; (ii) a controlled comparison of universal, hard-routed and soft-routed restoration including oracle versus predicted routing and an analysis of routing behaviour; (iii) a style-conditioned pix2pix model; and (iv) a containerised application that exposes all four models through ONNX Runtime.

II. Related Work

Denoising autoencoders. Vincent et al. [1] showed that reconstructing clean inputs from corrupted ones forces an autoencoder to learn robust features; convolutional variants such as DnCNN [2] and RED-Net [3] became strong restoration baselines. RED-Net and U-Net [4] rely on skip connections; we deliberately avoid them in Tasks 1–3 because unrestricted skips let the network copy the input around the bottleneck, which the assignment rules out and which would also copy impulse noise to the output.

Loss functions. Zhao et al. [5] showed that ℓ_2 training correlates poorly with perceived quality and that a mix of ℓ_1 and (MS-)SSIM [6] gives sharper, more structurally faithful restorations. This motivates our $\alpha\mathcal{L}_{L1} + (1-\alpha)(1-\text{SSIM})$ objective with α tuned by Optuna.

All-in-one restoration. AirNet [7] and PromptIR [8] restore several unknown degradations with one network by learning degradation representations, which is the universal setting of Task 1; our Task 2/3 systems make the degradation representation explicit through a classifier/gate.

Mixture of experts. Jacobs et al. [9] introduced gated expert mixtures; Shazeer et al. [10] and the Switch Transformer [11] showed that without an auxiliary load-balancing term the gate collapses onto a few experts. Our routing-balance regulariser follows this line; temperature-scaled softmax gating [12] controls sharpness.

Conditional GANs. GANs [13], conditional GANs [14] and pix2pix [15] (U-Net generator, PatchGAN discriminator, $\lambda_{L1} = 100$) form the basis of Task 4. FS2K [17] is

TABLE I
Corruption configuration

Condition	Training (random per load)	Test: low / medium / high
Clean	identity	identity
Salt & pepper	$p \sim \mathcal{U}(0.02, 0.15)$, black/white 50:50	$p = 0.03/0.08/0.15$
Gaussian blur	$k \in \{3, 5, 7\}$, $\sigma \sim \mathcal{U}(0.5, 2.5)$	$(3, 0.7)/(5, 1.5)/(7, 2.5)$
Occlusion	1-3 black boxes, 10-35% area	1/2/3 boxes, 10/20/35%

a recent facial-sketch benchmark with three sketch styles. Categorical conditioning through learned embeddings follows [14], [16].

Hyper-parameter optimisation and deployment. Optuna [19] provides define-by-run search with the TPE sampler [20] and median pruning. ONNX [22] and ONNX Runtime decouple training from serving.

III. Data and Runtime Corruption Pipeline

A. Datasets and splits

Oxford-IIIT Pet [18] provides 3680 official train+val and 3669 official test images (37 breeds; labels unused). The train+val set was split 80/20 with seed 42 into 2944 training and 736 validation images; the same split is used for Tasks 1-3 and the test set was only touched by the final evaluation script. All images are converted to RGB and resized to 128×128 with area interpolation and cached as a compressed array.

B. Corruptions

Table I lists the corruption definitions. During training, `RuntimeCorruptionDataset.__getitem__` draws a new condition and a new severity every time an image is loaded, from fresh OS entropy (so workers and epochs never repeat). The condition is chosen with equal probability; for the classifier and the MoE a `BalancedBatchSampler` additionally forces exactly $B/4$ samples of each class per batch. Salt-and-pepper replaces the selected pixels (all three channels) with black or white with probability $\frac{1}{2}$ each. Occlusion masks are placed without overlap, which makes the joint coverage equal to the sum of the rectangle areas, and a placement is accepted only if the coverage is within $\pm 1\%$ of the sampled target, so the 10-35% constraint is satisfied exactly (measured training range: 10.9-34.2%).

C. Deterministic manifests

Validation and test corruptions are generated once by `build_manifests` and stored as JSON with the condition, severity bucket, parameters (probability, kernel/ σ , rectangle coordinates, coverage) and the per-image seed that regenerates the salt-and-pepper mask. The validation manifest assigns one balanced condition per image with severities drawn from the training ranges; the test manifest contains, for each of the 3669 test images, the clean image plus the three corruptions at the three fixed severities (10 entries per image, 36690 in total).

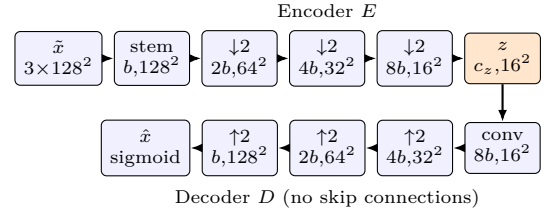


Fig. 1. Universal DAE $\hat{x} = D(E(\tilde{x}))$. Each block is two 3×3 conv-BN-LeakyReLU layers; $\downarrow 2$ = stride-2 conv, $\uparrow 2$ = bilinear upsampling. The orange 1×1 -conv latent is the bottleneck. The same architecture is reused for the three Task-2 specialists.

IV. Task 1: Universal Denoising Autoencoder

A. Architecture

The encoder (Fig. 1) is a stem of two 3×3 conv-BN-LeakyReLU layers followed by three stride-2 stages, so the spatial size goes $128 \rightarrow 64 \rightarrow 32 \rightarrow 16$ while channels grow $b \rightarrow 2b \rightarrow 4b \rightarrow 8b$. A 1×1 convolution then compresses the $8b$ feature maps into a latent tensor of $16 \times 16 \times c_z$; this is the only path to the decoder (no skip connections). The decoder mirrors the encoder using bilinear upsampling followed by convolution, which avoids the checkerboard artefacts of transposed convolutions [21], and ends with a sigmoid. With the selected $c_z = 16$ the latent has 4,096 values, a $12\times$ reduction relative to the 49,152-dimensional input, so the network is forced to learn a compressed representation rather than an identity map. Spatial dropout before and after the bottleneck regularises the latent.

We considered a fully-connected latent vector but chose a convolutional one on design grounds: flattening discards the spatial layout, a dense $8b \cdot 16^2 \times d$ projection would contain tens of millions of parameters for a 3.7k-image training set, and dense image decoders are known to produce blurry outputs, whereas a narrow 1×1 -conv bottleneck keeps locality at a comparable compression ratio with very few parameters. We also considered limited skip connections and did not use them: a high-resolution skip would give the decoder a direct path for impulse noise and occluder edges, so the model could minimise the loss on clean inputs by copying instead of learning a representation, which is exactly what the bottleneck requirement is meant to prevent.

B. Loss and training

The model minimises $\mathcal{L}_{UDAE} = \alpha \mathcal{L}_{L1}(x, \hat{x}) + (1 - \alpha)(1 - \text{SSIM}(x, \hat{x}))$ with an 11×11 Gaussian-window SSIM ($\sigma = 1.5$) implemented in PyTorch and computed in FP32 even under mixed precision. We use AdamW (weight decay 10^{-5}), cosine learning-rate decay, automatic mixed precision and horizontal-flip augmentation of the clean image before corruption, and keep the weights of the epoch with the best validation objective

$$S = \frac{1}{2} \text{SSIM} + \frac{1}{2} \min(\text{PSNR}, 40)/40, \quad (1)$$

TABLE II
Task 1 Optuna search space and selected configuration

Hyper-parameter	Range	Selected
Learning rate	$[10^{-4}, 3 \cdot 10^{-3}]$ log	7.3e-04
Batch size	{16, 32, 64}	16
Bottleneck channels c_z	{8, 16, 32}	16
Base channels b	{32, 48, 64}	64
Dropout	[0, 0.3]	0.012
Loss weight α	[0.5, 0.95]	0.557

TABLE III
Test PSNR (dB) by input condition

Condition	Input	T1	T2-Oracle	T2-Pred	T3-MoE
clean	∞	28.40	∞	∞	58.48
salt-pepper	16.39	28.23	28.15	28.15	28.63
blur	27.86	27.37	27.39	27.39	28.12
occlusion	13.57	22.37	22.80	22.81	22.63

which combines reconstruction quality (PSNR) and structure (SSIM) on comparable [0, 1] scales.

C. Optuna study

Table II gives the search space. The TPE sampler (seed 42) proposed 14 trials of 6 epochs each; a median pruner (3 start-up trials, 1 warm-up epoch) stopped 8 unpromising trials. The best trial (#13, $S = 0.6685$) was retrained for 45 epochs. Fig. 5 shows the optimisation histories and parameter importances.

D. Results

Tables III–VI report PSNR and SSIM on the full official test set (3669 images $\times$ 10 conditions) for all restoration systems; the “Input” column is the corrupted image itself, i.e. the no-restoration baseline. Fig. 2 shows twelve representative examples and Fig. 3 four failure cases; Fig. 4 shows the training curves.

Per corruption. The universal DAE raises the average PSNR over all corrupted inputs from 19.27 to 25.99 dB and SSIM from 0.633 to 0.817. The gain is largest for salt-and-pepper noise (16.4 $\rightarrow$ 28.2 dB, SSIM 0.38 $\rightarrow$ 0.87): impulses are statistically very different from natural image content and are removed almost completely at every severity, so the output quality is nearly independent of p (28.4/28.3/28.0 dB for low/medium/high). Occlusion is the hardest case (13.6 $\rightarrow$ 22.4 dB): the network fills the holes with a smooth, colour-consistent guess (Fig. 2, bottom rows) but cannot hallucinate texture, so the error map is concentrated inside the masks and grows with the occluded area (24.6/22.4/20.1 dB).

The cost of the bottleneck. Clean inputs come out at 28.4 dB / 0.875 SSIM, not as identical copies: the $12\times$ compressed latent cannot transmit all high-frequency detail, so every output is slightly smoothed. This sets a ceiling of roughly 28–28.5 dB that is visible in every row of Table V. Consequently, for mild blur (low severity, input

TABLE IV
Test SSIM by input condition

Condition	Input	T1	T2-Oracle	T2-Pred	T3-MoE
clean	1.000	0.875	1.000	0.999	0.999
salt-pepper	0.380	0.868	0.855	0.855	0.873
blur	0.812	0.825	0.822	0.822	0.843
occlusion	0.708	0.759	0.748	0.748	0.797

TABLE V
Test PSNR (dB) by corruption and severity

Condition	Input	T1	T2-Oracle	T2-Pred	T3-MoE
blur / high	24.53	25.59	26.00	26.00	26.14
blur / low	32.15	28.55	28.11	28.11	29.73
blur / medium	26.89	27.96	28.06	28.06	28.48
clean / none	∞	28.40	∞	∞	58.48
occlusion / high	10.73	20.11	20.77	20.77	20.56
occlusion / low	16.68	24.58	24.79	24.81	24.48
occlusion / medium	13.30	22.42	22.85	22.85	22.84
salt-pepper / high	13.15	28.00	27.94	27.94	28.26
salt-pepper / low	20.15	28.39	28.31	28.31	29.01
salt-pepper / medium	15.88	28.30	28.20	28.20	28.61

32.1 dB) the DAE makes the image measurably worse in PSNR (28.6 dB; worse than the input for 99.9% of these test images), while for medium and high blur it helps (26.9 $\rightarrow$ 28.0 and 24.5 $\rightarrow$ 25.6 dB). The same effect lowers SSIM for low-severity occlusion (0.861 $\rightarrow$ 0.822): most of the image was already perfect and is smoothed, while only 10% was restored. This is the expected trade-off of a genuine bottleneck without skip connections, and it motivates the identity branch used in Tasks 2 and 3.

Optuna. Eight of the 14 trials were pruned by the median rule, six of them after only 2–4 of the 6 epochs, which saved 25 of the 84 trial-epochs ($\approx 30\%$). The best configuration uses the widest encoder ($b = 64$), a 16-channel bottleneck, a small batch (16) and almost no dropout; the selected $\alpha = 0.557$ is markedly lower than the suggested 0.8, i.e. the search preferred a stronger SSIM term, consistent with [5]. Training was still improving at epoch 45 (the best validation objective was reached at the last epoch), so a longer schedule would likely help.

Failure cases (Fig. 3). (1) Image borders: a photograph with a thin white/black frame and low-severity salt-and-pepper noise drops from 19.5 to 17.2 dB, because the decoder cannot reproduce a sharp one-pixel frame through the bottleneck and smears it into a coloured halo. (2) Saturated flat backgrounds: for a cat on a uniform purple background with mild blur, the DAE introduces colour shifts in the flat region (37.2 $\rightarrow$ 24.8 dB) — the network has learned to “sharpen”, which creates artefacts where there is nothing to sharpen. (3) Occluders on black backgrounds: when a black mask falls on an already black background, the corruption is invisible, yet the network in-paints a bright, wrong colour into the dark region (38.1 $\rightarrow$ 12.3 dB); it has learned the shortcut “large black area $\Rightarrow$ fill it”, a direct consequence of the black-occluder definition. (4) High-frequency structure under strong blur: the slats of a

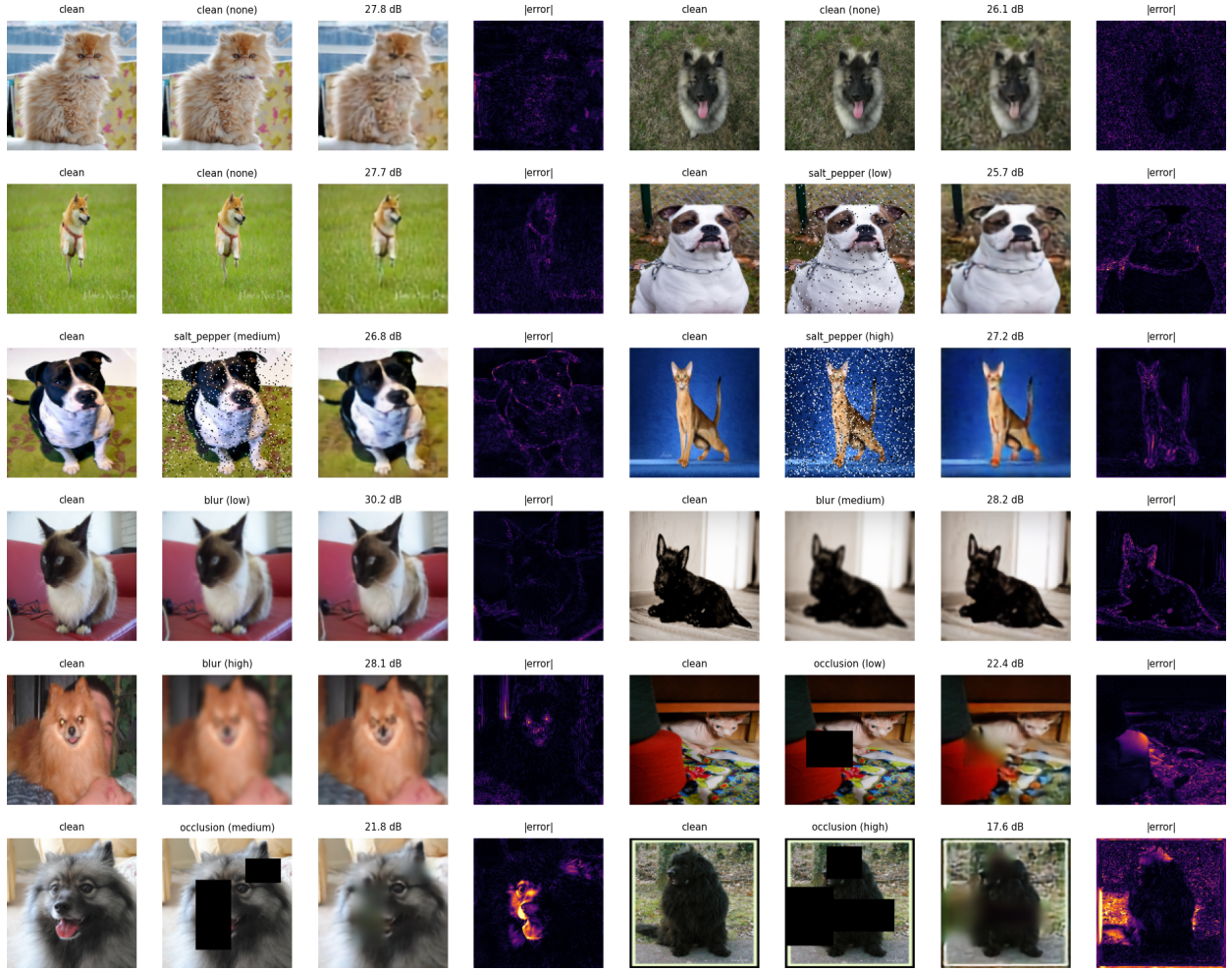


Fig. 2. Task 1: twelve representative test examples (clean target, corrupted input, reconstruction with PSNR, absolute error map). Rows: clean inputs, then salt-and-pepper, blur and occlusion at low/medium/high severity.

TABLE VI
Test SSIM by corruption and severity

Condition	Input	T1	T2-Oracle	T2-Pred	T3-MoE
blur / high	0.695	0.749	0.760	0.760	0.765
blur / low	0.937	0.872	0.857	0.857	0.901
blur / medium	0.804	0.854	0.848	0.848	0.862
clean / none	1.000	0.875	1.000	0.999	0.999
occlusion / high	0.536	0.686	0.682	0.682	0.705
occlusion / low	0.861	0.822	0.805	0.805	0.881
occlusion / medium	0.726	0.768	0.756	0.756	0.804
salt-pepper / high	0.202	0.860	0.848	0.848	0.862
salt-pepper / low	0.601	0.874	0.860	0.860	0.884
salt-pepper / medium	0.337	0.870	0.857	0.857	0.873

fence blurred with (7, 2.5) cannot be recovered (18.1 dB); the information is destroyed by the blur and the bottleneck cannot hallucinate a periodic pattern.

V. Task 2: Classification and Hard-Routed Specialists

A. Corruption classifier

The classifier C stacks 4–5 blocks of two 3×3 conv–BN–ReLU layers and 2×2 max-pooling. The first block works

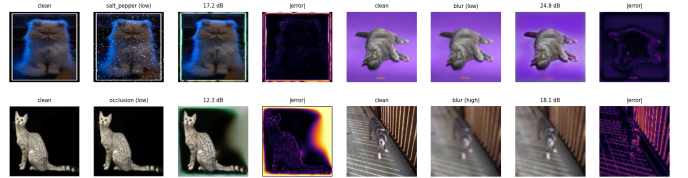


Fig. 3. Task 1 failure cases (lowest PSNR gain per corruption type and lowest SSIM overall).

at full resolution because both blur and isolated impulses are high-frequency cues that early striding would erase. Global average and global max pooling are concatenated before dropout and a linear layer: max pooling preserves strong local evidence such as a single black rectangle or a few saturated pixels that average pooling would dilute. It is trained with cross-entropy on balanced batches (exactly $B/4$ samples per class), so the classifier cannot exploit a class prior. Optuna tuned the learning rate, batch size $\{32, 64, 128\}$, channel configuration $\{\text{small, medium,}$

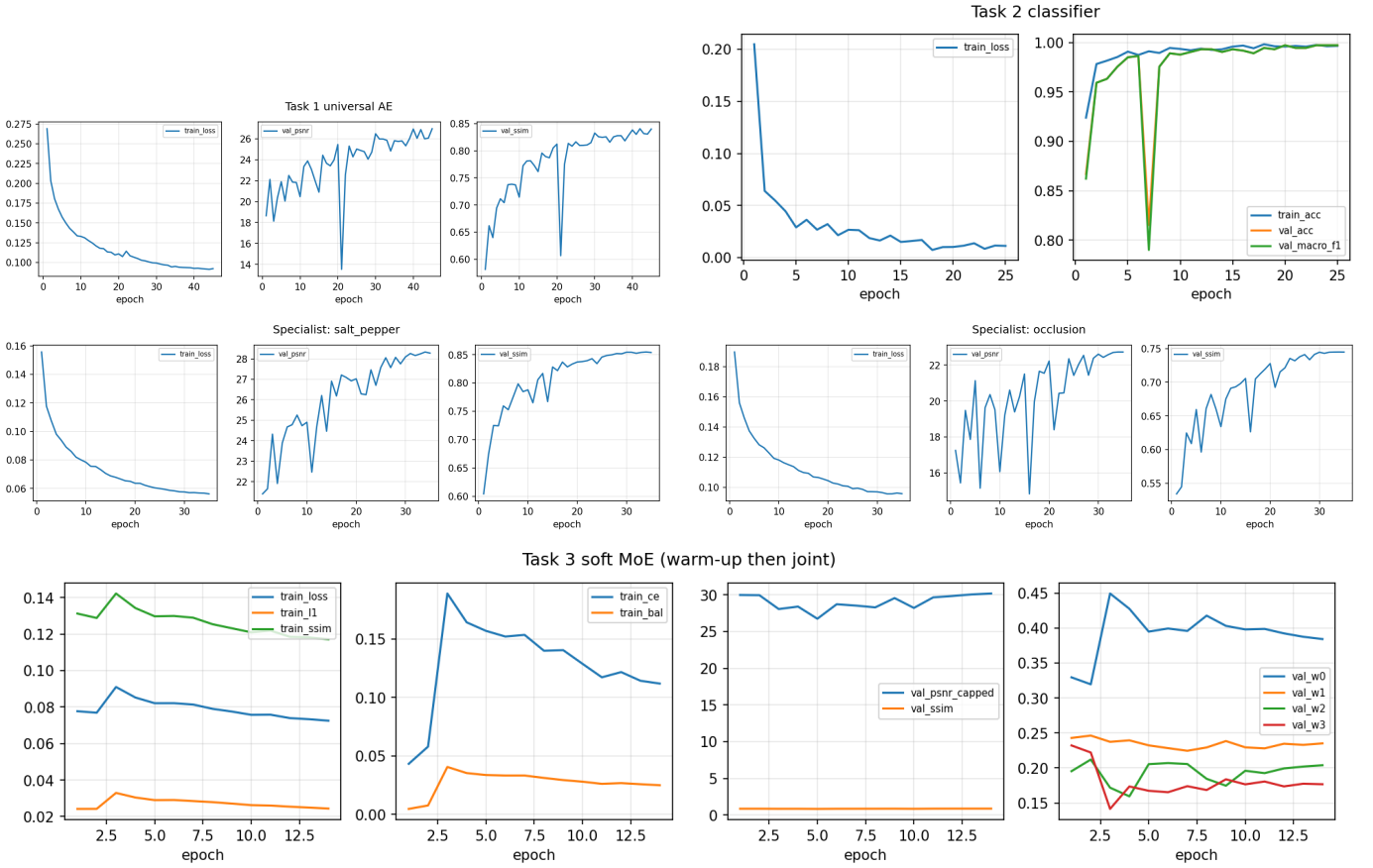


Fig. 4. Training and validation curves of the final runs: Task 1 universal DAE, Task 2 classifier, two of the Task 2 specialists (the blur specialist behaves alike), and Task 3 soft MoE (epochs 1–2: warm-up with frozen experts; afterwards joint fine-tuning; mean routing weights on the validation set in the last panel).

large}, dropout $[0, 0.5]$ and weight decay $[10^{-6}, 10^{-2}]$ (log), maximising validation macro-F1 (10 trials; best: $\text{lr}=7.6\text{e-}04$, $\text{batch}=32$, $\text{channels}=\text{large}$, $\text{dropout}=0.136$, $\text{wd}=0.002$).

B. Specialists

The three specialists reuse the Task-1 architecture with independent parameters; each sees only its own corruption (the classes argument of the runtime dataset) and reconstructs the same clean target. To keep the search affordable, one shared Optuna study selected a common architecture: each trial trains all three specialists briefly and returns the mean of their validation objectives, so the chosen configuration is good for all three rather than for one. The search covered learning rate, bottleneck channels, base channels, batch size and α (8 trials; best: $\text{lr}=2.4\text{e-}04$, $\text{batch}=16$, $c_z=32$, $b=48$, $\alpha=0.769$). The three specialists were then trained independently for 35 epochs.

C. Routing

At inference $r = \arg \max_k p_k$; $r = \text{clean}$ returns $\tilde{x}$ unchanged (identity bypass), otherwise only the selected specialist is executed (Fig. 8a). We evaluate oracle routing

(the manifest label selects the expert) and predicted routing (the classifier selects it).

D. Results

Classifier. On the validation manifest the classifier reaches 99.73 % accuracy and 0.997 macro-F1; on the full test set (36,690 inputs) it reaches 99.87 % accuracy, macro precision 0.998, macro recall 0.998 and macro-F1 0.998 (Table VII, Fig. 6). Salt-and-pepper is recognised perfectly; all errors involve the clean class: 0.4 % of clean images are predicted as blur and 0.2 % as occlusion, and 0.2 % of occluded images are predicted as clean. Accuracy is 100 % at medium and high severity and 99.8 % at low severity, i.e. errors only occur where the corruption is weakest.

Oracle vs. predicted routing. Because the classifier is almost perfect, predicted routing is practically identical to oracle routing: 26.12 vs. 26.11 dB and 0.808 vs. 0.808 SSIM over all corrupted inputs (Tables III–VI). Only 46 of 36,690 inputs (0.13 %) were misrouted. Interestingly, predicted routing is marginally better than oracle routing on low-severity occlusion (24.81 vs. 24.79 dB): in the 25 occlusion→clean errors the black mask lies on an already

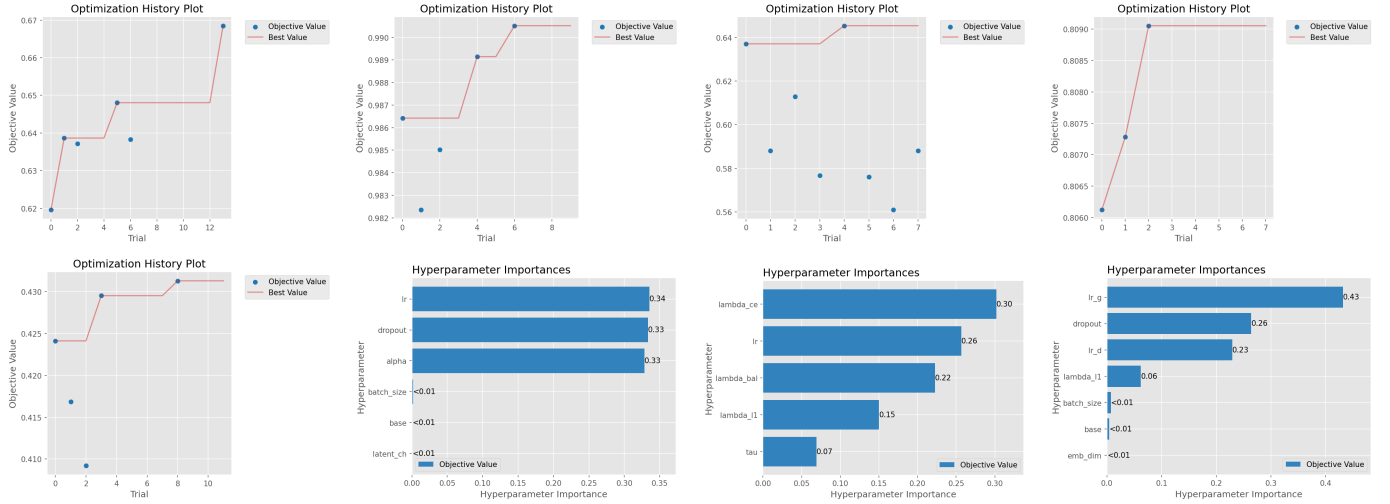


Fig. 5. Optuna studies. Top: optimisation histories of the Task 1 DAE, Task 2 classifier, Task 2 shared specialist search and Task 3 soft MoE. Bottom: Task 4 GAN history and fANOVA hyper-parameter importances for Tasks 1, 3 and 4 (importances use completed trials only).

TABLE VII
Classifier test metrics (36,690 test inputs)

Class	Precision	Recall	F1
Clean	0.993	0.994	0.994
Salt & pepper	1.000	1.000	1.000
Gaussian blur	0.999	1.000	0.999
Occlusion	0.999	0.998	0.999
Macro avg.	0.998	0.998	0.998
Accuracy	0.9987		

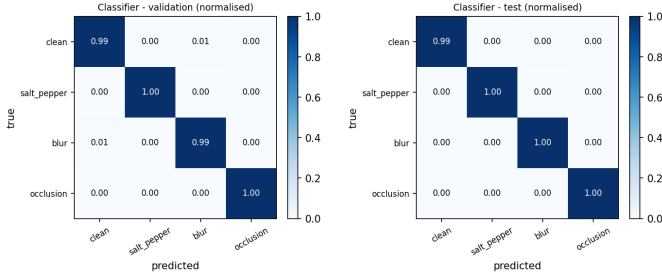


Fig. 6. Normalised four-class confusion matrices of the corruption classifier on the validation (left) and test (right) manifests.

black region, so the occlusion is effectively invisible and the identity bypass returns a nearly perfect image, whereas the occlusion specialist would have in-painted a wrong colour.

Misrouting failures (Fig. 7). The harmful errors are the 21 clean images sent to an expert, which lose 13.6 dB on average. They have a clear common cause: letter-boxed or framed photographs with black bars at the borders. The classifier interprets the black bars as occluders and routes the image to the occlusion specialist, which then “repairs” the bars by painting them with background colour (11.6–12.8 dB). The remaining clean→blur errors

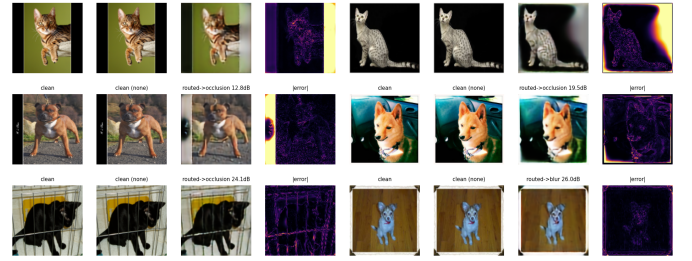


Fig. 7. Predicted-routing failures with the largest PSNR loss: clean images with black borders are routed to the occlusion expert, which paints over the borders.

are soft, low-contrast photographs that genuinely look slightly blurred. This shows that the occlusion definition (pure black rectangles) is ambiguous with naturally occurring black regions, and that a hard decision converts a small classification error into a large restoration error.

Specialists vs. universal model. The comparison with Task 1 is instructive. With an identity bypass, clean images are returned unchanged (100 dB / SSIM 1.0 under oracle routing; 99.6 dB / 0.999 predicted) instead of being smoothed to 28.4 dB, which is the single largest advantage of routing. On corrupted inputs, however, the specialists are not clearly better than the universal DAE: they win on occlusion PSNR (22.80 vs. 22.37 dB) and high blur (26.00 vs. 25.59 dB) but lose slightly on salt-and-pepper (28.15 vs. 28.23 dB) and on SSIM for every corruption (0.808 vs. 0.817 on average). Two factors explain this. First, both systems hit the same ≈ 28 dB ceiling imposed by the bottleneck, so specialisation cannot help where the bottleneck, not the task diversity, is the limit. Second, the shared specialist search preferred a smaller encoder ($b = 48$) and a larger L1 weight ($\alpha = 0.77$ vs. 0.56),

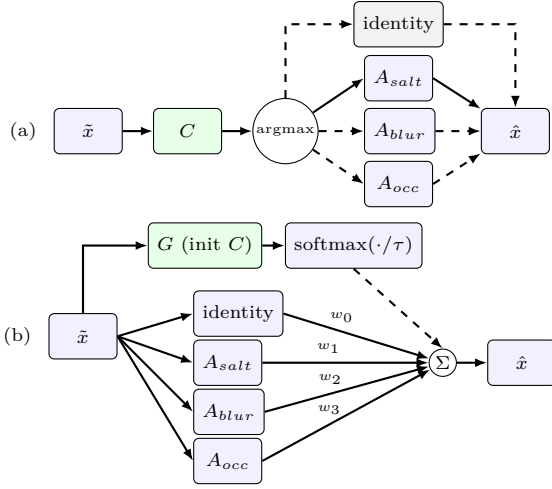


Fig. 8. (a) Hard routing: only the expert chosen by the classifier runs (dashed = not executed for this input). (b) Soft MoE: all branches run and are blended with gate weights w ; gate and experts are trained jointly.

which trades SSIM for PSNR. In other words, the universal network had enough capacity to learn all three corruptions jointly at this resolution.

VI. Task 3: Jointly Trained Soft Mixture-of-Experts

A. Model

The gate G has the classifier architecture and is initialised with the trained Task-2 classifier; the experts are initialised with the three trained specialists (Fig. 8b). For input $\tilde{x}$, $w = \text{softmax}(G(\tilde{x})/\tau)$ and $\hat{x} = w_0\tilde{x} + w_1A_{\text{salt}}(\tilde{x}) + w_2A_{\text{blur}}(\tilde{x}) + w_3A_{\text{occ}}(\tilde{x})$. Because the combination is differentiable, the reconstruction error back-propagates into both the gate and the experts. The whole pipeline, including the softmax, the temperature and the weighted sum, is exported as one ONNX graph with two outputs (image and weights).

B. Loss and two-stage training

$\mathcal{L}_{\text{MoE}} = \lambda_1\mathcal{L}_{L1} + \lambda_s(1 - \text{SSIM}) + \lambda_c\mathcal{L}_{CE}(G(\tilde{x}), y) + \lambda_b\mathcal{L}_{\text{balance}}$, with $\mathcal{L}_{\text{balance}} = \sum_{k=0}^3(\bar{w}_k - \frac{1}{4})^2$ computed on balanced batches, where the ideal average weight of each branch is exactly $\frac{1}{4}$. The cross-entropy term keeps the gate semantically tied to the corruption label, while the balance term (cf. the auxiliary load-balancing losses of [10], [11]) penalises collapse onto one branch. Note that, unlike in sparse MoE language models, here the balance term only acts on the batch mean; per-sample weights remain free to be sharp, which is what we want. Training starts with a 2-epoch warm-up in which the experts are frozen (parameters requires_grad=False, BN in eval mode) and only the gate is trained (lr = 10^{-4}); afterwards all parameters are fine-tuned jointly with a smaller, Optuna-selected learning rate for 12 epochs.

C. Optuna study

The joint learning rate [$10^{-5}, 3 \cdot 10^{-4}$] (log), $\tau \in [0.5, 2]$, $\lambda_c \in [0.01, 0.5]$ (log), $\lambda_b \in [10^{-3}, 0.1]$ (log) and the reconstruction weighting $\lambda_1 \in [0.5, 0.95]$ with $\lambda_s = 1 - \lambda_1$ were searched (8 trials). Besides median pruning we added a domain-specific pruning rule: a trial is stopped as soon as the mean validation weight of any branch falls below 0.02, i.e. when routing collapses (0 trials were pruned this way). Selected: $\lambda_1=0.509$, lr= $2.7\text{e-}04$, $\tau=1.749$, $\lambda_c=0.023$, $\lambda_b=0.002$.

D. Results

A model-selection pitfall and its fix. Our first MoE run (v1) selected checkpoints with S computed from the mean PSNR of the validation set. Because the identity branch returns clean images almost unchanged, their PSNR is ≈ 100 dB, and these few values dominated the mean: the validation mean PSNR fell from 39.3 to ≈ 36.8 dB as soon as joint fine-tuning started, although SSIM rose from 0.865 to 0.871. The criterion therefore kept the end of the warm-up (experts still frozen) as the “best” model, the v1 MoE was in effect hard routing with a re-trained gate (corrupted inputs 26.17 dB / 0.811 SSIM, routing entropy 0.07, 97 % of inputs with $\max_k w_k > 0.9$), and the joint fine-tuning had been discarded. We found this by inspecting the per-epoch history rather than only the final numbers. The fix caps PSNR at 40 dB per image before averaging; the whole MoE stage (Optuna search and final training) was then re-run (v2) from the same Task-2 checkpoints. All MoE results below are from v2; the v1 study is kept in the repository as evidence.

Reconstruction. With the corrected criterion the best checkpoint is the last joint epoch (Fig. 4, bottom): validation SSIM rises from 0.871 after warm-up to 0.884, so joint fine-tuning now clearly helps. On the test set the soft MoE is the best system on corrupted inputs, with 26.46 dB / 0.838 SSIM versus 25.99 dB / 0.817 (universal DAE) and 26.12 dB / 0.808 (predicted hard routing). It is best in SSIM for every corruption and severity (Table VI) and best in PSNR for all blur and salt-and-pepper levels. The largest gains appear exactly where the bottleneck hurt the other systems: low blur (29.7 dB / 0.901 vs. 28.1 / 0.857 for hard routing) and low occlusion (SSIM 0.881 vs. 0.805). Clean images keep near-identity quality (58.5 dB, SSIM 0.999). The only regression is occlusion PSNR at medium/high severity (20.56 vs. 20.77 dB at high severity), discussed below.

Routing behaviour (Figs. 9, 10). The gate remains semantically aligned with the corruption: the matching branch receives the largest weight for clean (0.96), salt-and-pepper (0.89–0.96), blur (0.65–0.87) and occlusion (0.56–0.73) inputs, and the argmax of w agrees with the true label for 96.7 % of test inputs. No expert is inactive (overall mean weights 0.27/0.28/0.25/0.20), and no expert dominates unrelated inputs: the salt-and-pepper, blur and occlusion experts receive on average only 0.0002,

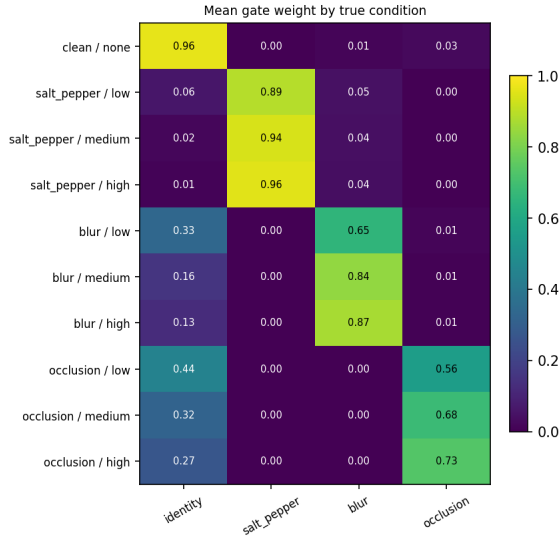


Fig. 9. Routing heatmap: mean gate weight of each branch for every true corruption and severity on the test set.

0.019 and 0.008 of the weight on inputs that are not of their type. The only branch used on unrelated inputs is the identity branch (0.19 on average for corrupted inputs) — and this is the key learned behaviour. The identity weight decreases monotonically with severity (blur: 0.33/0.16/0.13; occlusion: 0.44/0.32/0.27; salt-and-pepper: 0.06/0.02/0.01): for mild corruptions the gate blends a sizeable fraction of the (almost clean) input with the expert’s smoothed reconstruction, which restores the high-frequency detail that the bottleneck removes. This is effectively a learned, input-adaptive skip connection that hard routing cannot express. It already appears when only the gate is trained with the higher temperature of v2 (mean validation identity weight 0.32 after warm-up, where 0.25 would correspond to clean inputs only) and is strengthened by joint fine-tuning (0.38, Fig. 4), during which the experts adapt to being blended. The routing entropy increased from 0.07 (v1) to 0.45, and only 35% of inputs now have one branch with $w > 0.9$.

Dominant vs. distributed examples (Fig. 11). Strong salt-and-pepper noise is handled almost exclusively by the salt-and-pepper expert ($w_1 \approx 0.95$; the blur expert receives a small, consistent 0.03–0.05). The most distributed weights occur for low-severity salt-and-pepper on highly textured images (grass, flowers), where the gate splits $\approx 0.45/0.50$ between identity and the salt-and-pepper expert: the texture makes the sparse impulses hard to distinguish from image detail. This is also a failure mode of soft routing — blending 45% of the input re-introduces faint impulses, visible as dots in the error maps. The same mechanism explains the occlusion regression: blending 27–44% identity darkens the in-painted region again, which lowers PSNR inside large masks although SSIM, dominated by the unoccluded area, improves.

Mixed corruptions. To test the motivation of soft routing, we applied blur (5, 1.5) followed by salt-and-pepper $p = 0.08$ — a combination never seen in training — to 300 test images. The classifier labels all of them as salt-and-pepper and the gate also concentrates on that expert ($w_1 = 0.95$, $w_2 = 0.04$). The MoE (25.8 dB / 0.776) is only marginally better than hard routing (25.7 dB / 0.771), while the universal DAE is clearly best (27.0 dB / 0.821). Soft routing alone therefore does not solve compound degradations: the gate was never shown such inputs and the experts are applied in parallel, not in sequence, so no weighted sum of single-corruption experts can undo both corruptions. The universal model, which shares one representation across corruptions, generalises better here.

Optuna. Three of the eight v2 trials completed and five were pruned by the median rule; none had to be stopped by the routing-collapse rule, because the classifier initialisation and the cross-entropy term keep the gate aligned from the start. The search separates cleanly by temperature: all three completed trials used $\tau \geq 1.6$ and all five pruned trials $\tau \leq 1.29$. The selected configuration ($\lambda_1=0.509$, $\text{lr}=2.7\text{e-}04$, $\tau=1.749$, $\lambda_c=0.023$, $\lambda_b=0.002$) uses a learning rate near the top of the range, a high temperature (softer weights) and small classification and balance weights, which is consistent with the observed benefit of softer, blended routing.

VII. Task 4: Style-Conditioned Face-to-Sketch GAN

A. Data

We use the official FS2K split [17] (1058 training, 1046 test pairs). Fifteen percent of the training pairs, stratified by style with seed 42, form the validation set (899/159; validation styles 54/52/53). Photographs and sketches are resized to 128×128 ; the sketch of photo1/image0110 is sketch1/sketch0110, and we verified the pairing visually. Augmentation uses the pix2pix jitter (resize to 142, random 128 crop) and horizontal flips, applied with identical parameters to both images of a pair to preserve pixel correspondence.

B. Architecture

The generator is a 7-level U-Net [4], [15] ($128 \rightarrow 1$, channels b to $8b$, dropout in the three innermost decoder blocks). The style $s \in \{0, 1, 2\}$ is mapped by a learned `nn.Embedding` to $e_s \in \mathbb{R}^d$, which is broadcast spatially and concatenated (i) to the input photograph and (ii) to the 2×2 bottleneck features, so it controls both low-level stroke texture and global appearance. The discriminator is a PatchGAN [15] that receives the photograph, a real or generated sketch and its own style embedding map, and outputs a 14×14 grid of real/fake logits, each judging a 70×70 -pixel receptive field. Skip connections are appropriate here (unlike Tasks 1–3) because the sketch must align pixel-wise with the photograph.

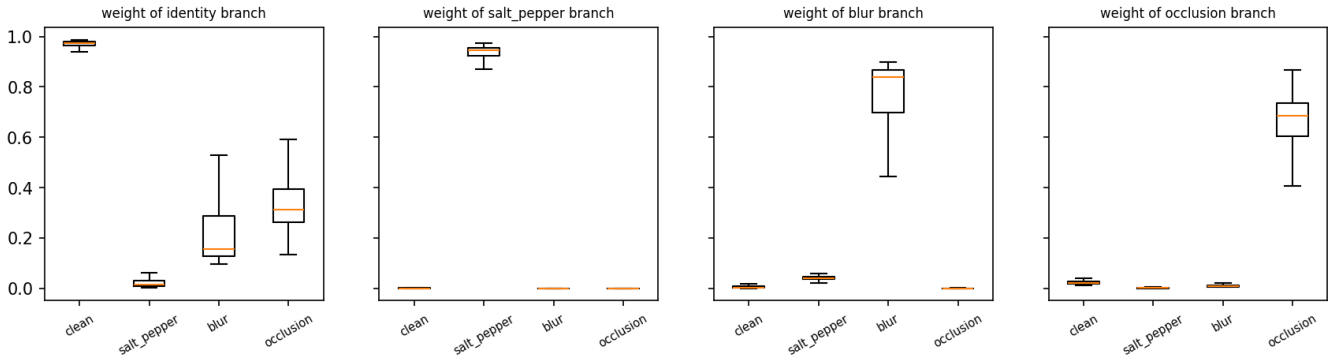


Fig. 10. Distribution of the four routing weights per true input condition (test set, box = inter-quartile range).

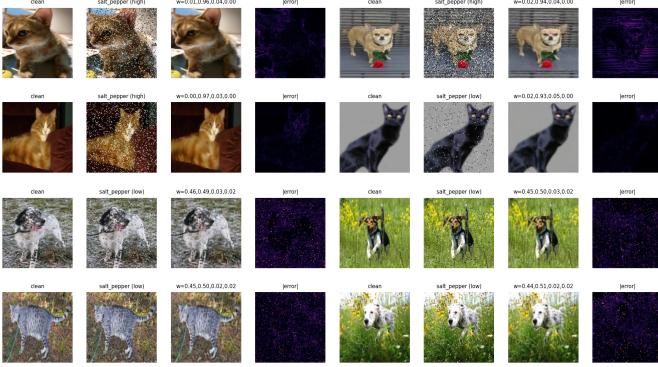


Fig. 11. Top: inputs where one expert dominates ($w > 0.9$). Bottom: inputs with the most distributed weights ($w = [w_0, w_1, w_2, w_3]$ in each title).

C. Objective and training

With BCE-with-logits, $\mathcal{L}_D = \frac{1}{2}[\text{BCE}(D(x, y, s), 1) + \text{BCE}(D(x, G(x, s), s), 0)]$ and $\mathcal{L}_G = \text{BCE}(D(x, G(x, s), s), 1) + \lambda_{L1}\|y - G(x, s)\|_1$. Adam ($\beta_1 = 0.5$) with a constant learning rate for the first half of training and linear decay to zero afterwards follows pix2pix. Discriminator real loss, discriminator fake loss, generator adversarial loss, generator L1 loss and validation PSNR/SSIM/L1 are logged separately every epoch, and a fixed grid of six validation photographs (two per style) rendered in all three styles is logged every 10 epochs.

D. Optuna study

Generator and discriminator learning rates, batch size $\{4, 8, 16\}$, base channels $\{32, 48, 64\}$, dropout $[0, 0.5]$, embedding dimension $\{4, 8, 16\}$ and $\lambda_{L1} \in [10, 200]$ (log) were searched with 12 trials of 30 epochs; the best configuration ($\text{lr}_G = 4.0\text{e-}04$, $\text{lr}_D = 4.5\text{e-}05$, batch=8, $b=48$, dropout=0.465, $d=16$, $\lambda_{L1}=111.071$) was retrained for 200 epochs. The validation objective is the same S as in Task 1.

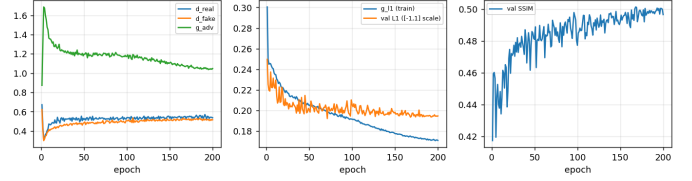


Fig. 12. Task 4 training: discriminator real/fake loss and generator adversarial loss (left), generator L1 on training and validation data (middle), validation SSIM (right).

E. Results

Training dynamics (Fig. 12). After a short transient in the first epochs, the discriminator real and fake losses settle at ≈ 0.53 and ≈ 0.51 and the generator adversarial loss decreases slowly from 1.2 to 1.05: neither network overpowers the other and there is no sign of mode collapse. The Optuna history (Fig. 5) shows that the reconstruction weight was the decisive hyper-parameter: all seven pruned trials used $\lambda_{L1} \leq 72$, all five completed ones $\lambda_{L1} \geq 78$, independent of the learning rates. Part of this preference is built into the selection criterion, which is pixel-based (SSIM/PSNR) and therefore rewards a strong L1 term; a perceptual criterion might favour a smaller λ_{L1} . Among the completed trials the fANOVA importance attributes most of the variance to lr_G (0.43), dropout (0.26) and lr_D (0.23) and almost none to λ_{L1} (0.06); this does not contradict the previous observation, because importance is computed on completed trials only, which all had a large λ_{L1} . The selected configuration uses $\lambda_{L1} = 111$ (close to the pix2pix default of 100), $\text{lr}_G = 4.0 \cdot 10^{-4}$ with a 9× smaller $\text{lr}_D = 4.5 \cdot 10^{-5}$, a large decoder dropout (0.47) and the largest embedding ($d = 16$). The training L1 keeps decreasing until epoch 200 while the validation L1 flattens after ≈ 120 epochs, indicating mild over-fitting; the best validation objective was reached at epoch 168, whose weights are used.

Test results. On the 1,046 official FS2K test pairs the generator reaches SSIM 0.480, PSNR 15.4 dB and L1 0.107 (images in $[0, 1]$). Results differ strongly by style: style 3

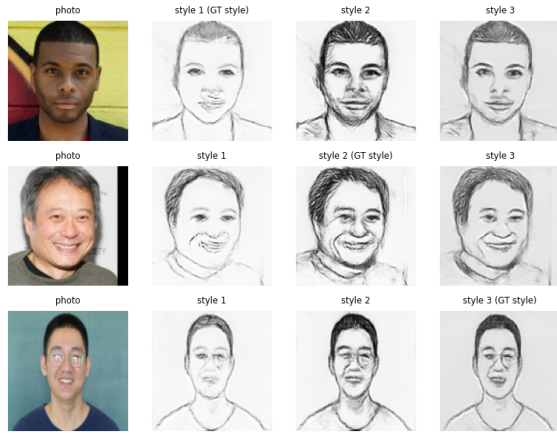


Fig. 13. Style control: the same test photograph rendered with each of the three style embeddings (“GT style” marks the style of the artist’s sketch).

(light pencil) 0.625 / 18.5 dB, style 1 (clean line drawing) 0.522 / 17.1 dB and style 2 (dense charcoal shading) 0.393 / 12.3 dB. Pixel metrics are a harsh measure for sketches: a stroke drawn two pixels away from the artist’s stroke is penalised twice, although perceptually it is equally valid, so these numbers should be read as relative rather than absolute quality indicators.

Style conditioning works. Fig. 13 renders the same test photograph with all three style embeddings. The outputs differ in exactly the way the FS2K styles differ: style 1 produces sparse contour lines, style 2 heavy dark hatching in hair and shadows, style 3 soft grey pencil shading with a light background. Since the photograph is identical in each row, these differences can only come from the embedding, which confirms that the condition is used by the generator and not ignored. Fig. 14 shows random test examples; identity, pose, glasses and hats are preserved, and facial features are aligned with the ground truth.

Failure cases (Fig. 15). All four lowest-SSIM test cases are style 2 portraits with long hair. The artist draws hair as a dense mass of long, high-contrast strokes, whereas the generator produces fewer, lighter and shorter strokes: under the L1 term, the safest prediction for a stroke whose exact position is uncertain is a light grey average, which the adversarial term only partly counteracts. Style 2 is also the hardest style overall, and the test set is imbalanced (619/381/46 images for styles 1/2/3), so the overall average is dominated by the two harder styles. A second, smaller failure mode is visible in Fig. 14: lips and nostrils are sometimes rendered with doubled or fragmented contours (“ghost” strokes) where the photograph has strong colour edges but the artist drew a single line.

Progress over training. The fixed validation grid logged every 10 epochs (W&B and Fig. 16) shows that after the first epoch the outputs are blurry, but their tone already depends on the style (style 2 is darker); by epoch 10 stroke-like textures appear, still with speckle noise; by

FS2K test: random examples (2 per style)



Fig. 14. FS2K test examples (two per style): photograph, generated sketch, artist’s sketch.

epoch 30 contours are clean and the three styles are clearly distinct; later epochs mostly sharpen strokes and remove background speckle.

VIII. Application and Deployment

A. Design process

The interface was designed first in Google Stitch (prompted with the four workspaces, the controls each needs and the information each must display). Stitch produced a dark Material-3 style dashboard with a fixed sidebar, a status footer, a top telemetry bar, a control card on the left and image panels plus stat tiles on the right (Fig. 17). The exported HTML is kept in design/stitch/. We implemented the design in React 18 + Tailwind CSS 4 by transferring the Stitch colour tokens (surface, primary,

FS2K test: lowest-SSIM cases

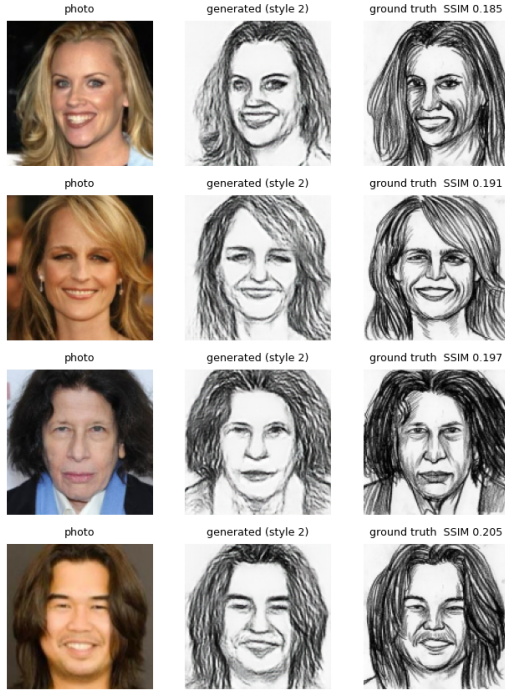


Fig. 15. Task 4 failure cases (lowest test SSIM): dense style-2 hair is under-drawn.



Fig. 16. The same validation photographs (rows) in styles 1–3 after epoch 1, 30 and 200.

tertiary, ...), typography (Geist, Inter, JetBrains Mono), Material Symbols icons, the gradient call-to-action and the card/panel structure into Tailwind theme variables and reusable components (Card, ImagePanel, Stat, Bars). Elements that Stitch invented but that would misrepresent the system (“A100 GPU”, “diffusion”, fake VRAM and checkpoint names) were replaced by real information from the back end (ONNX Runtime version, execution provider, number of loaded models, latency, uptime).

B. Architecture

Fig. 18 shows the deployment. docker compose up –build builds two images: a multi-stage Node 22 → nginx image that serves the compiled React bundle and reverse-proxies

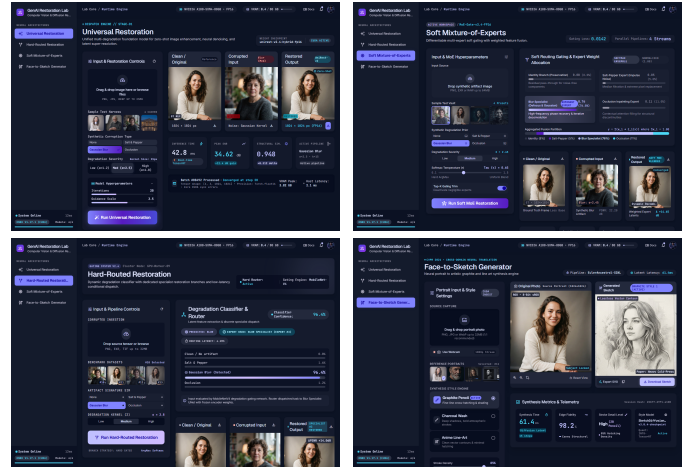


Fig. 17. Original Google Stitch designs (rendered from the exported HTML) for the four workspaces.

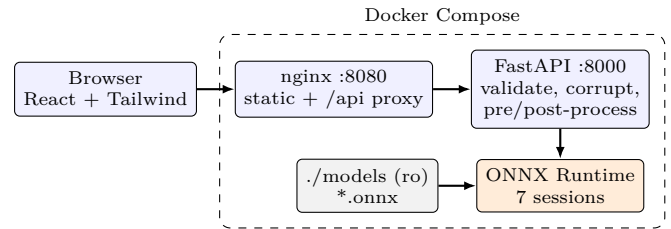


Fig. 18. Application architecture.

/api to the back end, and a Python 3.11 image running FastAPI with ONNX Runtime (CPU execution provider). The ONNX files are mounted read-only from ./models, which scripts/download_models.py fills from the GitHub release, so no Python script has to be run by hand inside the containers. A Docker health check on /api/health gates the start of the front end.

C. Back end

The FastAPI service exposes GET /api/health, GET /api/system, GET /api/samples, POST /api/restore/universal, /api/restore/hard, /api/restore/moe and /api/sketch. Uploads are validated (MIME type, 10 MB limit, decodability, EXIF orientation); sample paths are resolved and checked against the sample directory to prevent path traversal. A user can upload an already corrupted image (corruption=none) or apply any corruption at one of the three test severities with a chosen seed, using the same genai/corruptions.py module as training, so the app reproduces the test conditions exactly. When a clean reference exists, the response also contains PSNR/SSIM before and after restoration and an absolute-error map. The hard-routing endpoint runs the classifier, then either the identity bypass or exactly one specialist session, and returns the four probabilities, the predicted class, the chosen expert and separate classifier/expert latencies.

TABLE VIII

ONNX export: size and maximum absolute deviation from PyTorch

Model	Size (MB)	max $ \Delta $
universal_ae.onnx	27.1	7.7e-07
classifier.onnx	9.0	1.9e-06
specialist_salt_pepper.onnx	15.5	1.0e-06
specialist_blur.onnx	15.5	1.3e-06
specialist_occlusion.onnx	15.5	1.4e-06
soft_moe.onnx	55.6	2.4e-07
sketch_generator.onnx	90.2	2.0e-06

The MoE endpoint returns the four routing weights, their ranking and the routing entropy. Face photographs are centre-cropped to a square before resizing so that webcam frames are not distorted.

D. ONNX export and verification

All seven inference graphs (universal DAE, classifier with softmax, three specialists, the complete soft MoE and the sketch generator with its $[-1, 1] \leftrightarrow [0, 1]$ conversion folded in) were exported with opset 17 and a dynamic batch axis. Each exported model was run with ONNX Runtime on real test images plus random inputs and compared with PyTorch; Table VIII lists the maximum absolute deviation, which is at floating-point round-off level for every model. The results are stored in `onnx_verification.json` and shown on the application’s System page.

E. Interface

Fig. 19 shows the implemented workspaces. Every workspace shows the input, the output, a download button for every image, the inference time measured around the ONNX Runtime call, and the round-trip time measured in the browser. The Hard-Routed workspace adds the classifier probability bars, the predicted corruption, the selected expert and whether the routing was correct (when the true corruption is known). The Soft MoE workspace adds the four weight bars, a stacked contribution bar, the dominant branch and the routing entropy. The Face-to-Sketch workspace supports upload, sample selection and webcam capture, three style cards and a side-by-side view with download.

F. Experiment tracking

All Optuna trials (one W&B run each, grouped per study), the final training runs with their per-epoch losses and validation metrics, logged sample images, model checkpoints (as W&B artifacts) and the test-set evaluation tables and figures are recorded in the W&B project <https://wandb.ai/m-ibrahim-malik-national-university-of-computer-and-eme/genai-a1>. The Optuna studies themselves are stored as SQLite databases and JSON summaries in `results/optuna/`.

IX. Limitations

- **Bottleneck ceiling.** Without skip connections, all restoration models saturate at ≈ 28 dB on 128×128 images and slightly degrade mild corruptions (low blur, small occlusions). A restricted skip (e.g. one gated, low-resolution skip) or residual prediction would raise the ceiling and should be studied with an ablation.
- **Synthetic, single corruptions.** All corruptions are synthetic and applied one at a time. The models were not trained on combinations, real camera blur, JPEG artefacts or sensor noise; the mixed blur+noise experiment shows that the routed systems do not generalise to combinations.
- **Ambiguous occluder definition.** Pure black rectangles are indistinguishable from naturally black regions (letter-box bars, dark backgrounds), which causes most classifier errors and the worst restoration failures.
- **Search budget.** Each Optuna study used only 8–14 trials with short trial schedules (4–30 epochs) on two free Kaggle T4 GPUs; the Task 1 model was still improving at the end of training, and wider search spaces (architecture depth, schedulers) were not explored.
- **Evaluation of sketches.** Task 4 is evaluated with pixel metrics only. Perceptual or distributional metrics (LPIPS, FID) and a user study would better reflect sketch quality, and the strongly imbalanced FS2K test set (46 style-3 images) makes per-style comparisons noisy.
- **Validation objective.** Our first validation objective averaged PSNR before capping it, which let near-infinite identity outputs dominate model selection for the MoE (Section VI); the corrected objective caps PSNR per image. Other averaged metrics in the report should be read with the same caveat for clean inputs, where identity routing yields ≈ 100 dB.
- **Deployment.** Inference runs on CPU with ONNX Runtime in FP32 (tens to a few hundred ms per image); no quantisation, batching or GPU provider was used, and the app processes one image at a time at 128×128 .

X. Conclusion

We built and compared three restoration strategies on identical data and a fourth, generative image-to-image system, and deployed all of them as one containerised web application. The evidence supports four conclusions. (1) A single universal DAE with a genuine bottleneck learns all three corruptions (19.27→25.99 dB), but its bottleneck caps fidelity at ≈ 28 dB and degrades clean and mildly corrupted images. (2) Corruption classification is easy for these synthetic corruptions (99.87% test accuracy), so predicted hard routing matches oracle routing; its main benefit is the identity bypass for clean inputs, and its main

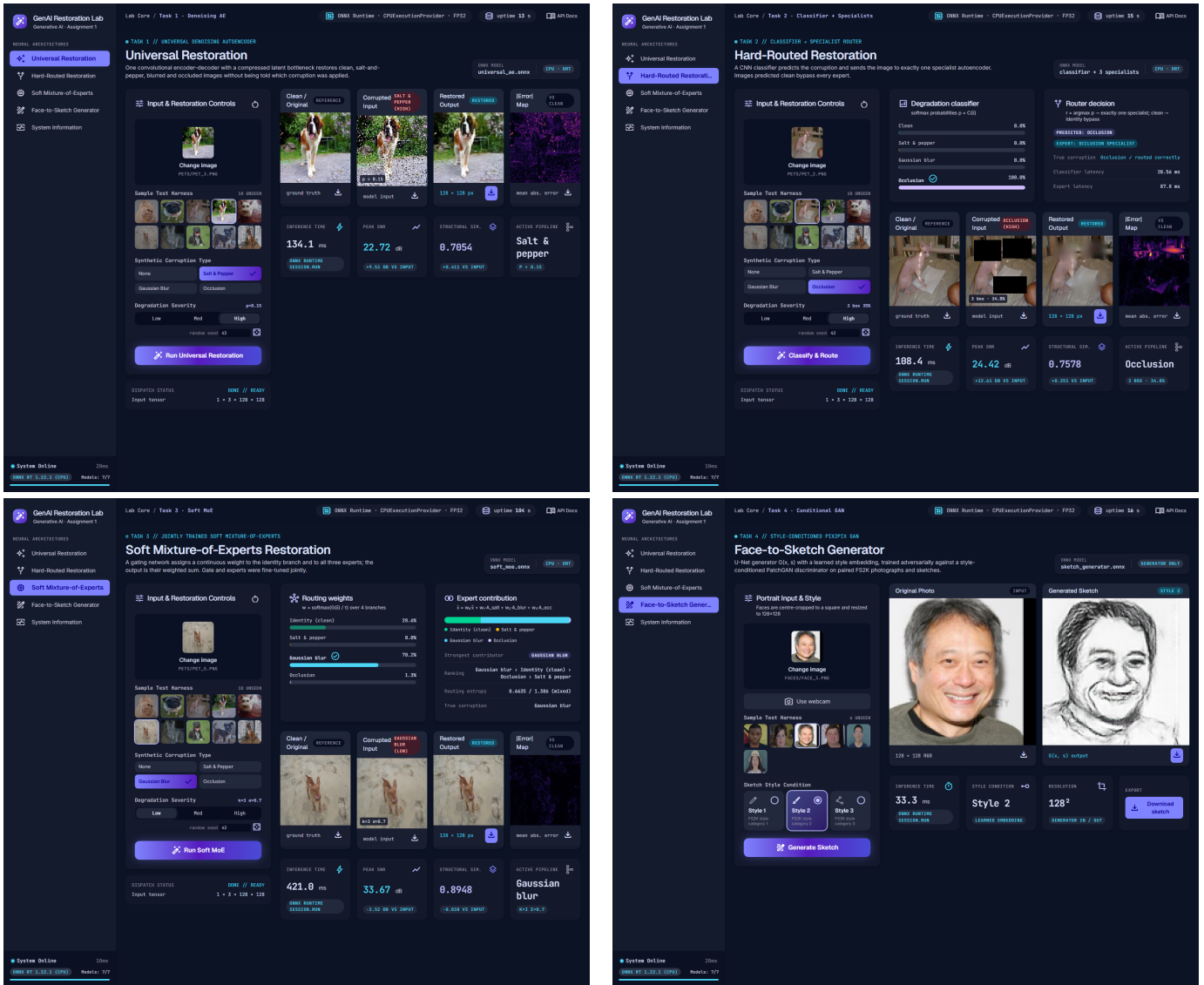


Fig. 19. The deployed application (running in Docker) on unseen test images: Universal Restoration, Hard-Routed Restoration, Soft Mixture-of-Experts Restoration and Face-to-Sketch Generator.

risk is that the rare errors — black borders mistaken for occluders — become severe restoration failures. Specialists were not better than the universal model on corrupted inputs, showing that the bottleneck, not task diversity, was the limiting factor. (3) The jointly trained soft MoE is the best system (26.46 dB / 0.838 SSIM on corrupted inputs) because the gate learned to blend the identity branch in proportion to corruption severity, an input-adaptive skip that hard routing cannot represent; this only became visible after we fixed a model-selection criterion that had been dominated by near-infinite identity PSNR. Soft routing did not, however, solve unseen compound corruptions, where the universal model generalised best. (4) The style-conditioned pix2pix model uses its learned embedding meaningfully — the same face is rendered in three distinct styles — with dense style-2 hair as

the main weakness. Future work includes gated skip connections in the experts, sequential rather than parallel expert composition for compound degradations, training on mixed and real corruptions, and perceptual metrics for sketch evaluation.

References

- [1] P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in Proc. ICML, 2008, pp. 1096–1103.
- [2] K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising,” IEEE Trans. Image Process., vol. 26, no. 7, pp. 3142–3155, 2017.
- [3] X.-J. Mao, C. Shen and Y.-B. Yang, “Image restoration using very deep convolutional encoder-decoder networks with symmetric skip connections,” in Proc. NeurIPS, 2016.

- [4] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in Proc. MIC-CAI, 2015, pp. 234–241.
- [5] H. Zhao, O. Gallo, I. Frosio and J. Kautz, “Loss functions for image restoration with neural networks,” IEEE Trans. Comput. Imaging, vol. 3, no. 1, pp. 47–57, 2017.
- [6] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” IEEE Trans. Image Process., vol. 13, no. 4, pp. 600–612, 2004.
- [7] B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, “All-in-one image restoration for unknown corruption,” in Proc. CVPR, 2022, pp. 17452–17462.
- [8] V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, “PromptIR: Prompting for all-in-one blind image restoration,” in Proc. NeurIPS, 2023.
- [9] R. A. Jacobs, M. I. Jordan, S. J. Nowlan and G. E. Hinton, “Adaptive mixtures of local experts,” Neural Comput., vol. 3, no. 1, pp. 79–87, 1991.
- [10] N. Shazeer et al., “Outrageously large neural networks: The sparsely-gated mixture-of-experts layer,” in Proc. ICLR, 2017.
- [11] W. Fedus, B. Zoph and N. Shazeer, “Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity,” J. Mach. Learn. Res., vol. 23, no. 120, pp. 1–39, 2022.
- [12] G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” arXiv:1503.02531, 2015.
- [13] I. Goodfellow et al., “Generative adversarial nets,” in Proc. NeurIPS, 2014.
- [14] M. Mirza and S. Osindero, “Conditional generative adversarial nets,” arXiv:1411.1784, 2014.
- [15] P. Isola, J.-Y. Zhu, T. Zhou and A. A. Efros, “Image-to-image translation with conditional adversarial networks,” in Proc. CVPR, 2017, pp. 1125–1134.
- [16] A. Odena, C. Olah and J. Shlens, “Conditional image synthesis with auxiliary classifier GANs,” in Proc. ICML, 2017.
- [17] D.-P. Fan et al., “Facial-sketch synthesis: A new challenge,” Mach. Intell. Res., vol. 19, pp. 257–287, 2022.
- [18] O. M. Parkhi, A. Vedaldi, A. Zisserman and C. V. Jawahar, “Cats and dogs,” in Proc. CVPR, 2012.
- [19] T. Akiba, S. Sano, T. Yanase, T. Ohta and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in Proc. KDD, 2019.
- [20] J. Bergstra, R. Bardenet, Y. Bengio and B. Kégl, “Algorithms for hyper-parameter optimization,” in Proc. NeurIPS, 2011.
- [21] A. Odena, V. Dumoulin and C. Olah, “Deconvolution and checkerboard artifacts,” Distill, 2016.
- [22] ONNX Community, “Open Neural Network Exchange,” <https://onnx.ai>, accessed 2026.

Appendix A

Use of AI Tools

Tool. Claude Code (Anthropic, Claude Opus model) was used as a coding assistant inside VS Code. Google Stitch was used for the interface design, as required by the assignment.

Tasks it was used for. (i) Reading the assignment and planning the work under a tight schedule; (ii) writing the first version of the training package (data pipeline, corruption functions, models, training loops, Optuna objectives, evaluation and ONNX export), the FastAPI back end, the React front end, the Dockerfiles and the Compose file; (iii) translating the Stitch export into React/Tailwind components; (iv) drafting the structure and the methodological text of this report, including the TikZ diagrams; (v) looking up dataset details (FS2K folder structure, annotation format and official split script).

How the outputs were tested and corrected.

- End-to-end smoke test: before using GPU time, the complete pipeline (Optuna, training, evaluation, figures, ONNX export) was run on CPU with synthetic data and a “smoke” budget. This exposed a crash when the batch size exceeded the number of samples (empty epochs and a division by zero), which was fixed in the batch sampler and data loader.
- Corruption checks: corrupted test images were inspected visually for all three severities (Section III), and the occlusion coverage was measured over 50 placements per severity (10.0 %, 20.0 %, 35.0 % ± 0.1 %) and over 2,000 training samples (10.9–34.2 %).
- Data pairing: FS2K photograph/sketch pairs were plotted side by side to confirm that the loader pairs files correctly; the stratified split was verified by printing the style counts.
- ONNX: every exported model is compared numerically with PyTorch (Table VIII).
- API: every endpoint was called with valid inputs, with a non-image file (rejected with HTTP 415) and with a path-traversal sample name (rejected with HTTP 404).
- Interface: headless-browser screenshots were compared with the Stitch renderings; invented hardware details from the Stitch mock-up were replaced by real values from the back end.
- Inspection of training histories: reading the per-epoch MoE history (not only the final metrics) revealed that the AI-written validation objective let near-infinite identity PSNR dominate model selection; the objective was corrected and the MoE stage re-run (Section VI).
- Fact-checking the text: every quantitative statement drafted with AI assistance was checked against the result files; three draft statements (the cause of pruned GAN trials, the compute saved by pruning, and when the identity blending in the MoE emerges) were wrong and were corrected.
- Numbers in the report are not typed by hand: scripts/make_report_assets.py generates them from the result files written by the evaluation scripts.

All hyper-parameter values reported here were selected by Optuna, not by the assistant. I reviewed the generated code and can explain and modify every component.